

LAPORAN PRAKTIKUM

JOBSHEET 9

STACK



Disusun oleh:

NAURA FADHILLA ADITYA PUTRI

254107020007

TI-1C

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

A. Percobaan 1: Mahasiswa Mengumpulkan Tugas

1. Berikut ialah hasil pemrograman Mahasiswa20.java

```
Minggu9 > J Mahasiswa20.java > Language Support for Java(TM) by Red Hat > {} Minggu9
1  package Minggu9;
2
3  public class Mahasiswa20 {
4      String nim;
5      String nama;
6      String kelas;
7      int nilai;
8
9      public Mahasiswa20() {
10     }
11
12
13     public Mahasiswa20(String nim, String nama, String kelas) {
14         this.nim = nim;
15         this.nama = nama;
16         this.kelas = kelas;
17         nilai = -1;
18     }
19
20     // method tugasDinilai
21     void tugasDinilai(int nilai) {
22         this.nilai = nilai;
23     }
24
25
26 }
27
```

2. Berikut ialah hasil pemrograman Mahasiswa 20.java

```
Minggu9 > J StackTugasMahasiswa20.java > Language Support for Java(TM) by Red Hat > {} Minggu9
1  package Minggu9;
2  public class StackTugasMahasiswa20 {
3      Mahasiswa20[] stack;
4      int top;
5      int size;
6
7      public StackTugasMahasiswa20(int size) {
8          this.size = size;
9          stack = new Mahasiswa20[size];
10         top = -1;
11     }
12
13     public boolean isFull() {
14         if (top == size - 1) {
15             return true;
16         } else {
17             return false;
18         }
19     }
20
21     public boolean isEmpty() {
22         if (top == -1) {
23             return true;
24         } else {
25             return false;
26         }
27     }
28
29     public void push(Mahasiswa20 mhs) {
30         if (!isFull()) {
31             top++;
32             stack[top] = mhs;
33         } else {
34             System.out.println(x: "Stack penuh! Tidak bisa menambahkan tugas lagi.");
35         }
36     }
37 }
```

```

Minggu9 > J StackTugasMahasiswa20.java > Language Support for Java(TM) by Red Hat > {} Minggu9
2 public class StackTugasMahasiswa20 {
38     public Mahasiswa20 pop() {
39         if (!isEmpty()) {
40             Mahasiswa20 mhs = stack[top];
41             top--;
42             return mhs;
43         } else {
44             System.out.println(x: "Stack kosong! Tidak ada tugas yang bisa diambil.");
45             return null;
46         }
47     }
48
49     public Mahasiswa20 peek() {
50         if (!isEmpty()) {
51             return stack[top];
52         } else {
53             System.out.println(x: "Stack kosong! Tidak ada tugas yang bisa dilihat.");
54             return null;
55         }
56     }
57
58     public void print() {
59         for (int i = 0; i <= top; i++) {
60             System.out.println(stack[i].nama + "\t" + stack[i].nim + "\t" + stack[i].kelas);
61         }
62         System.out.println(x: "");
63     }
64 }
65

```

3. Berikut ialah hasil pemograman MahasiswaDemo20.java

```

Minggu9 > J MahasiswaDemo20.java > Language Support for Java(TM) by Red Hat > MahasiswaDemo20 > main(String[])
1 package Minggu9;
2 import java.util.Scanner;
3
4 public class MahasiswaDemo20 {
5     Run main | Debug main | Run | Debug
6     public static void main(String[] args) {
7         Scanner scan = new Scanner(System.in);
8         // Instansiasi stack dengan kapasitas 5
9         StackTugasMahasiswa20 stack = new StackTugasMahasiswa20(size: 5);
10        int pilih;
11
12        do {
13            System.out.println(x: "\nMenu:");
14            System.out.println(x: "1. Mengumpulkan Tugas");
15            System.out.println(x: "2. Menilai Tugas");
16            System.out.println(x: "3. Melihat Tugas Teratas");
17            System.out.println(x: "4. Melihat Daftar Tugas");
18            System.out.print(s: "Pilih: ");
19            pilih = scan.nextInt();
20            scan.nextLine();
21
22            switch (pilih) {
23                case 1:
24                    System.out.print(s: "Nama: ");
25                    String nama = scan.nextLine();
26                    System.out.print(s: "NIM: ");
27                    String nim = scan.nextLine();
28                    System.out.print(s: "Kelas: ");
29                    String kelas = scan.nextLine();
30
31                    Mahasiswa20 mhs = new Mahasiswa20(nim, nama, kelas);
32                    stack.push(mhs);
33                    System.out.printf(format: "Tugas %s berhasil dikumpulkan\n", mhs.nama);
34                    break;
35
36                case 2:
37                    Mahasiswa20 dinilai = stack.pop();
38                    if (dinilai != null) {
39                        System.out.println("Menilai tugas dari " + dinilai.nama);
40                        System.out.print(s: "Masukkan nilai (0-100): ");
41                        int nilai = scan.nextInt();
42                        dinilai.tugasDinilai(nilai);
43                        System.out.printf(format: "Nilai Tugas %s adalah %d\n", dinilai.nama, nilai);
44                    }
45                    break;

```

```

Minggu9 > J MahasiswaDemo20.java > Language Support for Java(TM) by Red Hat > MahasiswaDemo20 > main(String[])
4   public class MahasiswaDemo20 {
5       public static void main(String[] args) {
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46         case 3:
47             Mahasiswa20 lihat = stack.peek();
48             if (lihat != null) {
49                 System.out.println("Tugas terakhir dikumpulkan oleh " + lihat.nama);
50             }
51             break;
52
53         case 4:
54             System.out.println(x: "Daftar semua tugas");
55             System.out.println(x: "Nama\tNIM\tKelas");
56             stack.print();
57             break;
58
59         default:
60             if (pilih < 1 || pilih > 4) {
61                 System.out.println(x: "Pilihan tidak valid.");
62             }
63     }
64     while (pilih >= 1 && pilih <= 4);
65 }
66 }

```

4. Hasil Output program

```

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 1
Nama: Dila
NIM: 1001
Kelas: 1A
Tugas Dila berhasil dikumpulkan

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 1
Nama: Erik
NIM: 1002
Kelas: 1B
Tugas Erik berhasil dikumpulkan

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 3
Tugas terakhir dikumpulkan oleh Erik

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 1
Nama: Tika
NIM: 1003
Kelas: 1C
Tugas Tika berhasil dikumpulkan

```

```

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 4
Daftar semua tugas
Nama      NIM      Kelas
Dila      1001      1A
Erik      1002      1B
Tika      1003      1C

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 2
Menilai tugas dari Tika
Masukkan nilai (0-100): 87
Nilai Tugas Tika adalah 87

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 4
Daftar semua tugas
Nama      NIM      Kelas
Dila      1001      1A
Erik      1002      1B

```

B. 2.1.3 Pertanyaan

1. Lakukan perbaikan pada kode program, sehingga keluaran yang dihasilkan sama dengan verifikasi hasil percobaan! Bagian mana yang perlu diperbaiki?

Jawab: Bagian yang perlu diperbaiki agar tampilannya sama dengan percobaan di halaman 6 adalah **metode print()**. Stack bekerja dengan prinsip **LIFO** (*Last In First Out*), sehingga daftar tugas seharusnya ditampilkan dari yang paling atas (top) ke bawah (0) agar tugas terbaru muncul pertama.

```

public void print() {
    // perbaikan kode agar LIFO (last in first out) sesuai dengan konsep stack
    for ([int i = top; i >= 0; i--]) {
        System.out.println(stack[i].nama + "\t" + stack[i].nim + "\t" + stack[i].kelas);
    }
    System.out.println(x: "");
}
}

```

2. Berapa banyak data tugas mahasiswa yang dapat ditampung di dalam Stack? Tunjukkan potongan kode programnya!

Jawab: Banyaknya data tugas yang dapat ditampung ditentukan saat **instansiasi objek** di class Main (MahasiswaDemo). Berdasarkan instruksi langkah ke-16, kapasitasnya adalah **5 data**.

```
Scanner scan = new Scanner(System.in);  
// Instansiasi stack dengan kapasitas 5  
StackTugasMahasiswa20 stack = new StackTugasMahasiswa20(size: 5);  
int pilih;
```

3. Mengapa perlu pengecekan kondisi !isFull() pada method push? Kalau kondisi if-else tersebut dihapus, apa dampaknya?

Jawab:

- Untuk memastikan bahwa masih ada ruang kosong di dalam array sebelum menambah data baru.
- Dampaknya jika dihapus: Jika kondisi if (!isFull()) dihapus, program akan terus mencoba menambah nilai top dan mengakses indeks array yang tidak ada. Hal ini akan menyebabkan error java.lang.ArrayIndexOutOfBoundsException saat stack sudah penuh.

4. Modifikasi kode program pada class MahasiswaDemo dan StackTugasMahasiswa sehingga pengguna juga dapat melihat mahasiswa yang pertama kali mengumpulkan tugas melalui operasi lihat tugas terbawah!

Jawab:

```
// No. 4 modifikasi liat tugas terbawah (data yang pertama kali masuk)  
public Mahasiswa20 peekBottom() {  
    if (!isEmpty()) {  
        return stack[0]; // melihat tugas terbawah (data yang pertama kali masuk)  
    } else {  
        System.out.println(x: "Stack kosong! Tidak ada tugas yang bisa dilihat.");  
        return null;  
    }  
}
```

```
case 5: // No. 4 modifikasi liat tugas terbawah (data yang pertama kali masuk)  
    Mahasiswa20 lihatBawah = stack.peekBottom();  
    if (lihatBawah != null) {  
        System.out.println("Tugas terbawah dikumpulkan oleh " + lihatBawah.nama);  
    }  
    break;
```

5. Tambahkan method untuk dapat menghitung berapa banyak tugas yang sudah dikumpulkan saat ini, serta tambahkan operasi menunya!

Jawab:

```
// no 5 modifikasi hitung jumlah tugas saat ini
public int jmlTugas() {
    return top + 1; // jumlah tugas saat ini adalah indeks top + 1
}
```

```
case 6: // no 5 modifikasi hitung jumlah tugas saat ini
    System.out.println("Jumlah tugas saat ini: " + stack.jmlTugas());
    break;

default:
    if (pilih < 1 || pilih > 6) {
        System.out.println(x: "Pilihan tidak valid.");
    }
}
} while (pilih >= 1 && pilih <= 6);|
```

C. Percobaan 2: Konversi Nilai Tugas ke Biner

1. Berikut buat file baru bernama StackKonversi.20java

```
package Mingguke9;

public class StackKonversi20 {
    int[] tumpukanBiner;
    int size;
    int top;

    public StackKonversi20() {
        this.size = 32; // asumsi 32 bit
        tumpukanBiner = new int[size];
        top = -1;
    }

    public boolean isEmpty() {
        return top == -1;
    }

    public boolean isFull() {
        return top == size - 1;
    }

    public void push(int data) {
        if (isFull()) {
            System.out.println(x: "Stack penuh! Tidak bisa menambahkan data lagi.");
        } else {
            top++;
            tumpukanBiner[top] = data;
        }
    }

    public int pop() {
        if (isEmpty()) {
            System.out.println(x: "Stack kosong! Tidak ada data yang bisa diambil.");
            return -1; // return -1 untuk menandakan stack kosong
        } else {
            int data = tumpukanBiner[top];
            top--;
            return data;
        }
    }
}
```

```
// konversi desimal ke biner
public String konversiDesimalkeBiner(int nilai) {
    StackKonversi20 stack = new StackKonversi20();
    // 2.2.3 modifikasi menjadi while (kode != 0)
    while (nilai != 0) {
        int sisa = nilai % 2;
        stack.push(sisa);
        nilai = nilai / 2;
    }
    String biner = new String();
    while (!stack.isEmpty()) {
        biner += stack.pop();
    }
    return biner;
}
```

Hasil output pemograman:

```
Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 1
Nama: tika
NIM: 1001
Kelas: 1C
Tugas tika berhasil dikumpulkan

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 2
Menilai tugas dari tika
Masukkan nilai (0-100): 78
Nilai Tugas tika adalah 78
Nilai dalam biner: 1001110
```

D. 2.2.3 Pertanyaan

1. Jelaskan alur kerja dari method konversiDesimalKeBiner!

Jawab:

- **Persiapan:** Method menerima input angka desimal (misalnya: 13). Sebuah stack kosong disiapkan untuk menampung angka biner.
- **Pembagian Berulang (Push):**

- Angka 13 dibagi 2, hasilnya **6** sisa **1**. Sisa 1 dimasukkan (*push*) ke stack.
- Angka 6 dibagi 2, hasilnya **3** sisa **0**. Sisa 0 dimasukkan (*push*) ke stack.
- Angka 3 dibagi 2, hasilnya **1** sisa **1**. Sisa 1 dimasukkan (*push*) ke stack.
- Angka 1 dibagi 2, hasilnya **0** sisa **1**. Sisa 1 dimasukkan (*push*) ke stack.

- **Pengambilan Data (Pop):**

- Karena Stack bersifat menumpuk, sisa terakhir yang masuk (angka 1 dari pembagian terakhir) berada di posisi paling atas.
- Saat kita melakukan `pop()`, angka-angka tersebut keluar dengan urutan terbalik dari saat dimasukkan.

- **Hasil Akhir:** Angka yang keluar disusun menjadi string: **1101**. Itulah hasil biner dari 13.

2. Pada method `konversiDesimalKeBiner`, ubah kondisi perulangan menjadi `while` (`kode != 0`), bagaimana hasilnya? Jelaskan alasannya!

Jawab:

```
// konversi desimal ke biner
public String konversiDesimalkeBiner(int nilai) {
    StackKonversi20 stack = new StackKonversi20();
    // 2.2.3 modifikasi menjadi while (kode != 0)
    while (nilai != 0) {
        int sisa = nilai % 2;
        stack.push(sisa);
        nilai = nilai / 2;
    }
    String biner = new String();
    while (!stack.isEmpty()) {
        biner += stack.pop();
    }
    return biner;
}
```

- Hasil: Hasil konversi desimal ke biner tetap sama (identik) dan tidak ada perubahan pada output program.
- Alasan: 1. Titik Henti Sama: Pada bilangan positif (0-100), kondisi `> 0` dan `!= 0` akan sama-sama berhenti tepat saat variabel mencapai angka 0.
- 2. Pembulatan Integer: Karena menggunakan tipe data `int`, hasil pembagian `$1 / 2$` akan menghasilkan 0, sehingga perulangan pasti berakhir.
- 3. Ekuivalensi Logika: Keduanya memiliki logika yang sama dalam memproses sisa pembagian (modulo) sebelum nilai benar-benar habis.

E. 2.4 Latihan Praktikum

1. Berikut ialah hasil pemograman Surat20.java

```
Mingguke9 > J Surat20.java > Language Support for Java(TM) by Red Hat > Surat20 > Surat20(String, String, String, char, int)
1  package Mingguke9;
2
3  public class Surat20 {
4      String idSurat;
5      String namaMhs;
6      String kelas;
7      char jenisSurat;
8      int durasi;
9
10     public Surat20() {
11     }
12
13     public Surat20(String idSurat, String namaMhs, String kelas, char jenisSurat, int durasi) {
14         this.idSurat = idSurat;
15         this.namaMhs = namaMhs;
16         this.kelas = kelas;
17         this.jenisSurat = jenisSurat;
18         this.durasi = durasi;
19     }
20 }
21
```

2. Berikut ialah hasil pemograman StackSurat20.java

```
Mingguke9 > J StackSurat20.java > ...
1  package Mingguke9;
2
3  public class StackSurat20 {
4      Surat20[] stack;
5      int top;
6      int size;
7
8      // stack untuk menyimpan surat yang masuk, dengan tipe data surat20
9      public StackSurat20(int size) {
10         this.size = size;
11         stack = new Surat20[size];
12         top = -1;
13     }
14
15     // method untuk mengecek apakah stack kosong
16     public boolean isEmpty() {
17         return top == -1;
18     }
19
20     // method untuk mengecek apakah stack penuh
21     public boolean isFull() {
22         return top == size - 1;
23     }
24
25     // method untuk menambahkan surat ke stack
26     public void push(Surat20 srt) {
27         if (!isFull()) {
28             top++;
29             stack[top] = srt;
30         } else {
31             System.out.println("Stack penuh! Tidak bisa menambahkan surat lagi.");
32         }
33     }
34 }
```

```

Mingguke9 > J StackSurat20.java > ...
3 public class StackSurat20 {
35 // method untuk mengambil surat dari stack
36 public Surat20 pop() {
37     if (!isEmpty()) {
38         Surat20 srt = stack[top];
39         top--;
40         return srt;
41     } else {
42         System.out.println(x: "Stack kosong! Tidak ada surat yang bisa diambil.");
43         return null;
44     }
45 }
46
47 // method untuk melihat surat teratas tanpa menghapusnya
48 public Surat20 peek() {
49     if (!isEmpty()) {
50         return stack[top];
51     } else {
52         return null;
53     }
54 }
55
56 // metod cari surat berdasarkan nama mahasiswa
57 public Surat20 cariSurat(String nama) {
58     boolean ditemukan = false;
59     for (int i = top; i >= 0; i--) {
60         if (stack[i].namaMhs.equalsIgnoreCase(nama)) {
61             System.out.println("Surat ditemukan pada tumpukan ke-" + (i+1));
62             System.out.println("ID: " + stack[i].idSurat + " | Jenis: " + stack[i].jenisSurat + " | Durasi: " + stack[i].durasi + " hari");
63             ditemukan = true;
64             break;
65         }
66     }
67     if (!ditemukan) {
68         System.out.println("Surat dengan nama mahasiswa " + nama + " tidak ditemukan.");
69     }
70 }
71 }
72

```

3. Berikut ialah hasil pemograman SuratMain20.java

```

Mingguke9 > J SuratMain20.java > SuratMain20 > main(String[] args)
1 package Mingguke9;
2 import java.util.Scanner;
3
4 public class SuratMain20 {
5     Run | Debug | Run main | Debug main
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8         StackSurat20 srt = new StackSurat20(size: 10);
9         int pilihan;
10
11         do {
12             System.out.println(x: "\nMenu Layanan Surat Izin");
13             System.out.println(x: "1. Terima surat Izin (Push)");
14             System.out.println(x: "2. Proses surat Izin (Pop)");
15             System.out.println(x: "3. Lihat surat Izin teratas (Peek)");
16             System.out.println(x: "4. Cari surat Izin berdasarkan nama mahasiswa");
17             System.out.println(x: "5. Keluar");
18             System.out.print(s: "Pilih: ");
19             pilihan = sc.nextInt();
20             sc.nextLine(); // membersihkan buffer
21
22             switch (pilihan) {
23                 case 1:
24                     System.out.print(s: "ID Surat: ");
25                     String idSurat = sc.nextLine();
26                     System.out.print(s: "Nama Mahasiswa: ");
27                     String namaMhs = sc.nextLine();
28                     System.out.print(s: "Kelas: ");
29                     String kelas = sc.nextLine();
30                     System.out.print(s: "Jenis Surat Izin (S/I): ");
31                     char jenisSurat = sc.next().charAt(index: 0);
32                     System.out.print(s: "Durasi (hari): ");
33                     int durasi = sc.nextInt();
34                     sc.nextLine(); // membersihkan buffer
35
36                     Surat20 s = new Surat20(idSurat, namaMhs, kelas, jenisSurat, durasi);
37                     srt.push(s);
38                     break;
39

```

```

Mingguke9 > J SuratMain20.java > SuratMain20 > main(String[] args)
4   public class SuratMain20 {
5       public static void main(String[] args) {
39           case 2:
40               Surat20 prosesSurat = srt.pop();
41               if (prosesSurat != null) {
42                   System.out.println("Memproses surat dari " + prosesSurat.namaMhs);
43               }
44               break;
45
46           case 3:
47               Surat20 lihatSurat = srt.peek();
48               if (lihatSurat != null) {
49                   System.out.println("Surat teratas adalah dari " + lihatSurat.namaMhs);
50               } else {
51                   System.out.println(x: "Belum ada surat masuk.");
52               }
53               break;
54
55           case 4:
56               System.out.print(s: "Masukkan nama mahasiswa yang ingin dicari: ");
57               String cariNama = sc.nextLine();
58               srt.cariSurat(cariNama);
59               break;
60
61       }
62   } while (pilihan != 5);
63 }
64 }
65

```

4. Hasil Output pemograman:

```

Menu Layanan Surat Izin
1. Terima surat Izin (Push)
2. Proses surat Izin (Pop)
3. Lihat surat Izin teratas (Peek)
4. Cari surat Izin berdasarkan nama mahasiswa
5. Keluar
Pilih: 1
ID Surat: 001
Nama Mahasiswa: Aldo
Kelas: 2C
Jenis Surat Izin (S/I): I
Durasi (hari): 2

Menu Layanan Surat Izin
1. Terima surat Izin (Push)
2. Proses surat Izin (Pop)
3. Lihat surat Izin teratas (Peek)
4. Cari surat Izin berdasarkan nama mahasiswa
5. Keluar
Pilih: 1
ID Surat: 002
Nama Mahasiswa: Rani
Kelas: 2A
Jenis Surat Izin (S/I): S
Durasi (hari): 3

Menu Layanan Surat Izin
1. Terima surat Izin (Push)
2. Proses surat Izin (Pop)
3. Lihat surat Izin teratas (Peek)
4. Cari surat Izin berdasarkan nama mahasiswa
5. Keluar
Pilih: 2
Memproses surat dari Rani

Menu Layanan Surat Izin
1. Terima surat Izin (Push)
2. Proses surat Izin (Pop)
3. Lihat surat Izin teratas (Peek)
4. Cari surat Izin berdasarkan nama mahasiswa
5. Keluar
Pilih: 3
Surat teratas adalah dari Aldo

```